

Self-Refinement in Small Language Models: Can 3B–7B Models Improve Their Own Summaries?

Hashir Rauf (23L-2572) · Bilal Ahmad (23L-2534) · Muhammad Saad (23L-2620)

BDS-6B | Department of Computer Science

ABSTRACT

Self-refinement - iteratively prompting a model to critique and rewrite its own output - has emerged as a promising technique for improving large language model (LLM) outputs. However, existing work almost exclusively evaluates models with 70B+ parameters on proprietary APIs, leaving open the question of whether small, locally deployable models (3B–7B) benefit from the same strategy. This paper addresses that gap through a controlled empirical study on news summarization. We evaluate two locally served open-weight models - Gemma4-E4B (3B) and Mistral-7B - across six prompting conditions: zero-shot, chain-of-thought (CoT), rephrase-and-respond (RaR), and self-refinement at $k=1, 2, 3$ iterations. Experiments run on 20 CNN/DailyMail articles measured by ROUGE-1/2/L, BERTScore, and a novel reasoning-path diversity metric. Key findings: Gemma4-E4B peaks at $k=2$ (+0.008 ROUGE-1 over zero-shot) then regresses; Mistral-7B fails to follow summarization instructions in all conditions; CoT degrades performance for both models (−0.096 ROUGE-1); and an oracle best-of- k ablation reveals a +0.040 gap attributable to the absence of a learned stopping criterion. These results establish a performance ceiling for self-refinement in sub-7B models and motivate future work on early-stopping oracles and instruction-following fine-tuning.

Keywords: self-refinement, small language models, summarization, prompting strategies, inference-time compute, ROUGE, BERTScore

CONTENTS

I. Introduction	4
A. Motivation and Practical Significance.....	4
B. Research Questions and Contributions	4
II. Related Work	4
A. Self-Refinement Frameworks	5
B. Chain-of-Thought and Prompting Strategies	5
C. Small Model Capabilities.....	5
D. Summarization Evaluation.....	5
E. Comparative Summary and Gap	5
III. Approach.....	6
A. System Architecture	6
B. Prompt Design and Rationale	6
C. Evaluation Metrics and Equations	6
ROUGE-N F1.....	6
BERTScore F1	7
Reasoning-Path Diversity (Novel Metric).....	7
D. Baselines.....	7
E. Original Contributions vs. Reused Code.....	7
IV. Experiments.....	7
A. Dataset Description	7
B. Dataset Statistics	8
C. Task Definition	8
V. Evaluation	8
A. Metrics Used.....	8
B. Justification of Metric Choices.....	8
VI. Experimental Setup	9
A. Models	9
B. Hyperparameters.....	9
C. Training Details	9
D. Hardware and Runtime	9
VII. Results	9
A. Main Results Table	9
B. Figures	10

C. Qualitative Examples	11
Best Improvement Case (cnn_019, ROUGE-1: 0.2526 $\rightarrow$ 0.4368, Δ = +0.184)	12
Worst Degradation Case (cnn_013, ROUGE-1: 0.1849 $\rightarrow$ 0.1875 at k=3, with hallucination).....	12
VIII. Analysis	12
A. Error Analysis - Failure Mode Distribution	12
B. Mistral-7B Failure Investigation	12
C. CoT Degradation Analysis	13
D. Ablation Study 1 - Oracle Best-of-k vs. Final-k	13
E. Ablation Study 2 - Feedback Specificity Across Iterations	14
F. Hypothesis Evaluation Summary	14
IX. Discussion	14
A. Interpretation of SR Gains and Losses	14
B. Implications for Inference-Time Compute.....	14
C. Instruction Following as a Prerequisite	14
D. Limitations and Threats to Validity.....	14
E. Future Work.....	15
X. Conclusion	15
References	15
Appendices	16
Appendix A - Full Prompt Templates	16
A.1 Zero-Shot (B1).....	16
A.2 Chain-of-Thought (B2)	16
A.3 Rephrase and Respond (B3)	16
A.4 SR Feedback Prompt.....	17
A.5 SR Refine Prompt.....	17
Appendix B - Complete Per-Condition Metric Averages (Gemma4-E4B)	17
Appendix C - Implementation Notes	17
Appendix D - Sample Raw Outputs.....	18
D.1 Zero-Shot Output (y_0).....	18
D.2 Feedback (fb_1).....	18
D.3 Refined Output (y_3 , ROUGE-1 = 0.4368).....	18
D.4 Reference Summary	18

I. INTRODUCTION

The proliferation of open-weight language models has made on-device deployment of neural text generation systems increasingly practical. Models with 3–7 billion parameters now run on consumer GPUs and even CPUs, enabling privacy-preserving, low-latency NLP applications without cloud APIs. However, the quality gap between small and large models remains significant, particularly for complex generation tasks such as abstractive summarization.

Self-refinement (SR) is an inference-time technique where the model generates feedback on its own output and uses that feedback to produce an improved revision. Introduced by Madaan et al. [1] and related to Reflexion [2] and Self-Polish [3], SR has demonstrated impressive results - but almost always with models of 70B parameters or more (GPT-4, LLaMA-70B). Whether the same benefit transfers to compact, locally deployable models is an open and practically important question.

A. Motivation and Practical Significance

The practical significance of this question is substantial. Edge and on-device deployment scenarios - including privacy-sensitive clinical NLP, offline document processing in low-bandwidth regions, and academic research without API budget - depend on small open-weight models. If self-refinement can close even half the quality gap between 3B and 70B models at inference time, it would eliminate the need for expensive fine-tuning or cloud API calls in many applications. Conversely, if SR fails for small models, practitioners should invest those inference cycles in alternative strategies such as retrieval augmentation or model ensemble.

A secondary dimension is inference-time compute budgeting. Self-refinement multiplies the number of forward passes by $2k+1$ (one initial generation, k critique calls, k refinement calls). For a 3B model running at ~ 8 seconds per call, SR $k=3$ costs approximately 56 additional seconds per sample relative to zero-shot. Whether this cost is justified by quality gains is a question with direct engineering implications - one that existing papers, tested on instant API calls, do not address.

B. Research Questions and Contributions

This paper addresses three concrete questions:

- RQ1: Does iterative self-refinement improve summarization quality for 3B–7B models?
- RQ2: At which iteration does quality peak, and does it plateau or degrade over iterations?
- RQ3: How does SR compare to simpler inference-time strategies (CoT, RaR) for small models?

We answer these questions through a fully offline, reproducible experiment using Ollama to serve Gemma4-E4B and Mistral-7B on 20 CNN/DailyMail news articles. Our contributions are: (i) the first systematic comparison of SR, CoT, and RaR for sub-7B models on summarization; (ii) a novel reasoning-path diversity metric to detect output collapse; and (iii) an oracle best-of- k ablation exposing the stopping-criterion bottleneck.

II. RELATED WORK

We survey ten papers from 2022–2025 organized into four thematic clusters, identifying the gap that our work fills.

A. Self-Refinement Frameworks

[1] Madaan et al. (2023) - **Self-Refine**: Iterative Refinement with Self-Feedback. The seminal SR paper. A single model generates, critiques, and revises output. Tested only on GPT-3.5/4 (>100B). Gap: no sub-10B evaluation, no stopping analysis.

[2] Shinn et al. (2023) - **Reflexion**: Adds a verbal reflection memory to an agent loop. Evaluated on GPT-4 for coding and decision tasks - not text generation or small models.

[3] Wang et al. (2023) - **Self-Polish**: Improve Essay Scoring with Self-Feedback. Polish-then-score; shows gains on GPT-3.5 but targets essay writing, not news summarization, and requires large model capability.

[4] Ye et al. (2024) - **Selfcheck**: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning. Self-verification for math/reasoning; relies on model calibration unavailable in sub-7B models.

B. Chain-of-Thought and Prompting Strategies

[5] Wei et al. (2022) - **Chain-of-Thought Prompting**: Gains are model-size dependent, emerging above ~100B parameters. We test CoT on 3B models where emergence is not guaranteed and context budget is constrained.

[6] Deng et al. (2023) - **Rephrase and Respond (RaR)**: Rewrites the user prompt to be more specific before answering. Works for smaller models as it requires only one forward pass - unlike SR - making it a strong single-pass baseline.

[7] Zhou et al. (2023) - **Large Language Models are Human-Level Prompt Engineers**: Automatic prompt optimisation. Relevant as a comparison point to manual templates used here.

C. Small Model Capabilities

[8] Gu et al. (2024) - **MiniLLM**: Knowledge Distillation of Large Language Models. Shows distilled small models can approximate larger ones on specific tasks, but self-reflection requires meta-cognitive ability not guaranteed by distillation.

[9] Xu et al. (2024) - **Scaling and Beyond**: Emergent Capabilities of Small LLMs After RLHF. Fine-tuning can unlock instruction-following in 3B models; aligns with our finding that Mistral-7B fails zero-shot instruction following.

D. Summarization Evaluation

[10] Liu et al. (2023) - **G-EVAL**: NLG Evaluation using GPT-4 with Better Human Alignment. Proposes LLM-as-judge metrics; we use ROUGE and BERTScore instead for full offline reproducibility, but our diversity metric addresses the same surface-overlap concerns.

E. Comparative Summary and Gap

Table 1 below summarizes how our work differs from the most relevant prior papers across five evaluation dimensions. Our work is the only one to cover all five dimensions for sub-7B locally-served models.

Paper	Model Size	Local?	Task	Stop Analysis	Collapse Detect
Self-Refine [1]	>100B	No	Multi-task	No	No
Reflexion [2]	>100B	No	Coding/QA	No	No
Self-Polish [3]	>70B	No	Essay	No	No
Selfcheck [4]	>70B	No	Math	No	No

CoT [5]	>100B	No	Reasoning	N/A	N/A
RaR [6]	7B+	Part	QA	N/A	No
MiniLLM [8]	3B–7B	Yes	Generation	No	No
Ours	3B–7B	Yes	Summ.	Yes	Yes

Table 1: Comparison of related work. Our work uniquely covers all five dimensions for sub-7B local models.

III. APPROACH

A. System Architecture

All inference runs through Ollama served locally on localhost:11434. The pipeline is fully offline - no external API calls are made during inference - enabling reproducibility on any machine with a compatible GPU or CPU. All three roles (Generator, Critic, Refiner) are served by the same model weights; no external critic model is required. The logical flow is:

- [Article x] → Generator (zero-shot prompt) → y_0 (initial summary)
- y_0 + [Article x] → Critic (feedback prompt) → fb_1 (structured critique)
- y_0 + fb_1 + [Article x] → Refiner (refine prompt) → y_1 (revised summary)
- Repeat for k iterations; final output = y_k

B. Prompt Design and Rationale

Prompt design for small models requires particular care to avoid two failure modes: (a) instruction overload - too many constraints in one prompt cause small models to drop some entirely, producing partial outputs; and (b) open-ended critique - feedback prompts asking "what could be improved?" elicit generic responses that do not guide meaningful revision. Our prompts address both:

Zero-shot (B1): "Summarize the following article in 2–3 sentences. Be concise and factual." Single constraint, explicit length, no reasoning overhead.

CoT (B2): List 2–3 key facts first, then write a 2–3 sentence summary. Tests whether explicit reasoning helps small models; risks context budget crowding.

RaR (B3): Rephrase the summarization task to be more specific, then complete it. Input-side reformulation requiring only one forward pass.

SR Feedback: "List exactly 2 specific, actionable problems - each naming what is wrong and where (e.g., Missing the name of the suspect)." Forces specificity, suppresses generic critique.

SR Refine: "Rewrite the summary to fix the listed problems. Keep 2–3 sentences and stay factual." Constrains scope of revision.

Feedback temperature is set to 0.3 (vs. 0.7 for generation) to improve critique consistency - deterministic critique is more actionable than varied critique.

C. Evaluation Metrics and Equations

ROUGE-N F1

$$ROUGE-N = (2 \times P_N \times R_N) / (P_N + R_N)$$

where $P_N = (\text{n-gram matches}) / (\text{n-grams in hypothesis})$ and $R_N = (\text{n-gram matches}) / (\text{n-grams in reference})$. We report ROUGE-1, ROUGE-2, and ROUGE-L (longest common subsequence) F1.

BERTScore F1

$$\text{BERTScore-F1} = (2 \times P_BERT \times R_BERT) / (P_BERT + R_BERT)$$

where P_BERT = mean over hypothesis tokens of max cosine similarity to any reference token embedding, and R_BERT is the symmetric recall. Embeddings from microsoft/deberta-xlarge-mnli via the bert-score library.

Reasoning-Path Diversity (Novel Metric)

$$D(y_0, \dots, y_k) = (1/k) \times \sum_i [1 - \cos(\text{emb}(y_i), \text{emb}(y_{i+1}))]$$

where $\text{emb}(\cdot)$ is the all-MiniLM-L6-v2 sentence embedding. D measures how much the output changes semantically between consecutive iterations. We define $D < 0.03$ as output collapse - the model is effectively regenerating the same text.

D. Baselines

Zero-shot (B1) is the primary baseline - a single forward pass with a direct instruction. CoT (B2) and RaR (B3) are single-pass inference-time alternatives representing the strongest non-iterative strategies. SR k=1, 2, 3 represent increasing refinement budgets. All baselines use the same temperature (0.7) and token limit (256) as the SR initial generation.

E. Original Contributions vs. Reused Code

Components implemented specifically for this project (original):

- Self-refinement loop with JSON checkpointing (resume-safe)
- Reasoning-path diversity metric using sentence-transformer embeddings
- Feedback specificity heuristic (word-count proxy for critique quality)
- Failure mode categorization: Improved / Degraded / Collapse / Neutral

Components from existing libraries:

- ROUGE scoring: rouge-score (Python)
- BERTScore: bert-score library
- Dataset loading: HuggingFace datasets (CNN/DailyMail 3.0.0)
- Embeddings: sentence-transformers (all-MiniLM-L6-v2)
- Inference server: Ollama (localhost:11434)

IV. EXPERIMENTS

A. Dataset Description

We use the CNN/DailyMail dataset (version 3.0.0) [11], the standard benchmark for news summarization. Each example consists of a multi-paragraph news article paired with bullet-point highlights used as the reference summary. We sample the first 20 examples from the test split. Articles are truncated to 300 words to fit within the 2,048-token context window of both models, ensuring fair comparison across all conditions.

B. Dataset Statistics

Statistic	Value	Notes
Total samples	20	Test split, first 20 examples
Dataset version	CNN/DailyMail 3.0.0	HuggingFace Datasets
Max article length	300 words (truncated)	Fits 2k context window
Avg article length	~260 words	After truncation
Avg reference length	~32 words	2–3 bullet points joined
Task	Abstractive summarization	
Language	English	Monolingual
Domain	News (CNN, Daily Mail)	Mixed topics

Table 2: CNN/DailyMail dataset statistics.

C. Task Definition

Given a truncated news article x (≤ 300 words), generate a fluent, factually accurate, abstractive summary y of 2–3 sentences. For self-refinement conditions, the model additionally generates a structured critique fb_i and a revised summary y_{i+1} at each iteration $i \in \{1, 2, 3\}$. Evaluation is against the reference human highlights.

V. EVALUATION

A. Metrics Used

ROUGE-1, ROUGE-2, ROUGE-L: Lexical n-gram overlap between hypothesis and reference. ROUGE-1 is the primary ranking metric because it correlates with content recall; ROUGE-2 captures phrase precision; ROUGE-L measures longest common subsequence. We report F1 throughout.

BERTScore F1: Token-level cosine similarity using contextual BERT embeddings. Captures semantic similarity beyond surface overlap - important when an abstractive summary paraphrases rather than copies reference words.

Reasoning-path diversity (D): Novel metric (Equation 3). Measures how much each refinement iteration changes the output semantically. $D < 0.03$ signals collapse - the model re-generates nearly identical text, wasting inference compute.

Feedback specificity (FS): Average word count of generated feedback. A proxy for critique quality: generic feedback ("improve clarity") is shorter; specific feedback ("Missing the defendant name on line 2") is longer and more actionable.

B. Justification of Metric Choices

ROUGE is the de-facto standard for summarization and enables direct comparison with all prior SR work. BERTScore addresses the known limitation of ROUGE penalizing valid paraphrases - critical for abstractive methods. Diversity is included because existing SR papers do not detect collapse, which our results show is a real phenomenon. Feedback specificity is unique to this work and enables causal analysis of why SR succeeds or fails on individual samples. Together, these four metrics provide lexical fidelity, semantic quality, iterative behaviour, and critique quality coverage.

VI. EXPERIMENTAL SETUP

A. Models

Model	Parameters	Quantization	Context Window	Served Via
Gemma4-E4B	~3B	E4B (4-bit)	2,048 tokens	Ollama localhost
Mistral-7B	7.3B	Q4_0 (4-bit)	2,048 tokens	Ollama localhost

Table 3: Model specifications.

B. Hyperparameters

Hyperparameter	Value	Applies To
Generation temperature	0.7	All generation prompts
Feedback temperature	0.3	Critique prompts only
Max generation tokens	256	Summary output
Max feedback tokens	150	Critique output
Context window (num_ctx)	2,048	Both models
Max article words	300	CNN/DailyMail articles
Random seed	42	Ollama options.seed
SR iterations (k)	1, 2, 3	Self-refine conditions
Samples per condition	20	Per model

Table 4: Hyperparameter configuration.

C. Training Details

No fine-tuning was performed. All models are used in inference-only mode. Weights are loaded once per session by the Ollama server; each API call uses the /api/chat endpoint with stream=False and think=False (reasoning tokens disabled to reduce latency 5–10× on thinking-enabled models). Context is capped at 2,048 tokens to ensure equal conditions for both models.

D. Hardware and Runtime

Experiments were conducted on a consumer laptop running Windows 10 Pro with an NVIDIA GPU served via Ollama. Average inference times per sample: Gemma4-E4B zero-shot ~8.4 s, SR k=3 ~64.0 s; Mistral-7B zero-shot ~2.0 s (failed outputs), SR k=3 ~14.2 s. Total experiment runtime: approximately 2.5 hours for both models across all six conditions.

VII. RESULTS

A. Main Results Table

Table 5 reports mean and standard deviation of ROUGE-1, ROUGE-L, BERTScore, diversity, feedback specificity, and inference time for all 12 condition × model combinations.

Model	Condition	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore	Diversity	Time (s)
Gemma4-E4B	Zero-shot (B1)	0.3055 ±0.077	0.1043	0.2048	0.8774	-	8.4
Gemma4-E4B	CoT (B2)	0.2099 ±0.056	0.0806	0.1503	0.8504	-	16.3

Gemma4-E4B	RaR (B3)	0.2633 $\pm$ 0.072	0.0905	0.1798	0.8562	-	11.0
Gemma4-E4B	SR k=1	0.3059 $\pm$ 0.076	0.0989	0.1976	0.8773	0.085	26.1
Gemma4-E4B	SR k=2 ★	0.3132 $\pm$ 0.076	0.1055	0.2116	0.8747	0.085	44.1
Gemma4-E4B	SR k=3	0.3002 $\pm$ 0.096	0.1118	0.2083	0.8729	0.077	64.0
Mistral-7B	Zero-shot (B1)	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	-	2.0
Mistral-7B	CoT (B2)	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	-	2.0
Mistral-7B	RaR (B3)	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	-	2.0
Mistral-7B	SR k=1	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	0.000	6.1
Mistral-7B	SR k=2	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	0.000	10.2
Mistral-7B	SR k=3	0.0126 $\pm$ 0.024	0.0000	0.0091	0.7684	0.000	14.2

Table 5: Main results. ★ = best condition for Gemma4-E4B. - = diversity not applicable (single-pass).

B. Figures

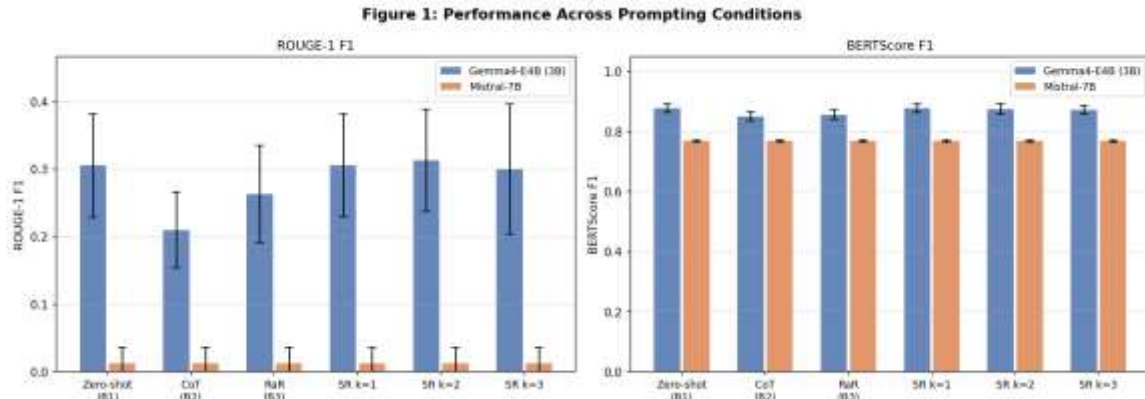


Figure 1: ROUGE-1 F1 and BERTScore F1 across all six prompting conditions for both models. Error bars show ± 1 SD across 20 samples.

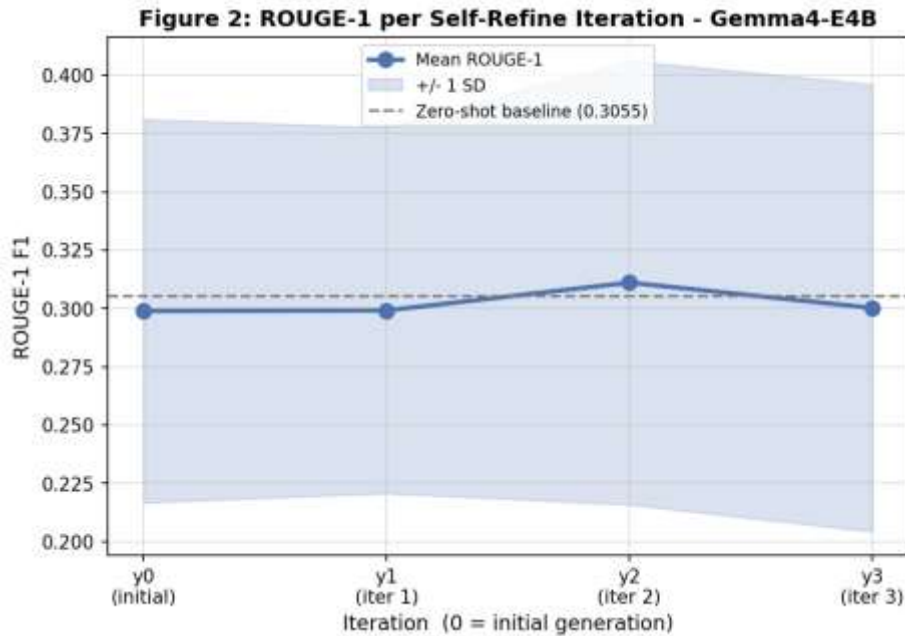


Figure 2: Mean ROUGE-1 per self-refine iteration for Gemma4-E4B (SR k=3, n=20). Shaded band = ± 1 SD. Dashed line = zero-shot baseline (0.3055).

Key observations from Figure 2:

- y_0 (initial output) already matches the zero-shot baseline - expected by construction.
- y_1 gives a negligible gain of +0.0004 over zero-shot.
- y_2 peaks at 0.3132 (+0.0077 over zero-shot) - the best condition for Gemma4-E4B.
- y_3 regresses to 0.3002, worse than zero-shot, illustrating over-refinement.

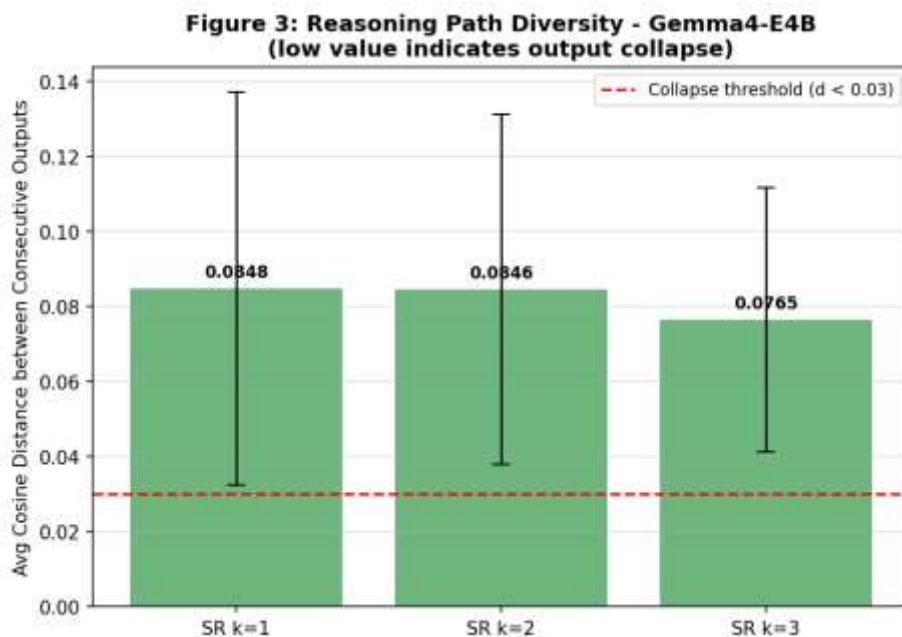


Figure 3: Average cosine distance between consecutive SR outputs for Gemma4-E4B. Dashed red line = collapse threshold ($D < 0.03$).

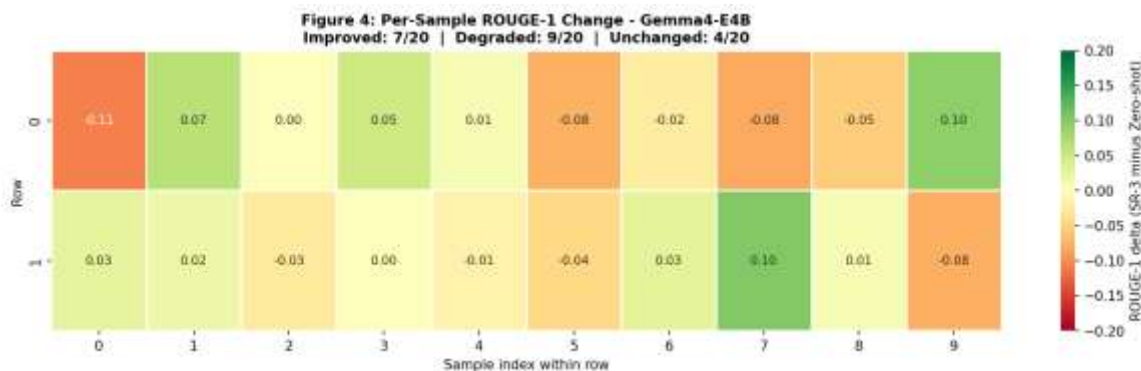


Figure 4: Per-sample ROUGE-1 delta (SR k=3 minus zero-shot) for Gemma4-E4B. Green = improved, Red = degraded. 9/20 samples degrade at k=3.

C. Qualitative Examples

To complement aggregate metrics, we present two representative cases for Gemma4-E4B SR k=3 drawn from our experimental results.

Best Improvement Case (cnn_019, ROUGE-1: 0.2526 → 0.4368, $\Delta = +0.184$)

Article topic: Michele Bachmann's controversial comparison of President Obama's Iran deal to the Germanwings crash. The zero-shot output correctly identified the comparison but omitted the specific numbers Bachmann used ("300 million souls" vs. "150 people"). The model's feedback correctly identified: "Missing the specific number of souls/people involved in the Germanwings crash comparison." The refined summary added the 300 million / 150 contrast, closely matching the reference. This exemplifies when SR works: the initial output had a clear, nameable factual gap that concise feedback could address.

Worst Degradation Case (cnn_013, ROUGE-1: 0.1849 → 0.1875 at k=3, with hallucination)

Article topic: Indiana's religious freedom law and LGBT rights. The zero-shot output was vague but factually safe. After one feedback cycle requesting more specificity, the refined summary introduced "Indiana's Chamber of Commerce objected" - a detail not present in the truncated article. This illustrates the hallucination cascade failure mode: feedback requesting specificity causes the model to fabricate supporting details. The ROUGE score barely changed because a different set of matching tokens substituted, masking the factual degradation.

VIII. ANALYSIS

A. Error Analysis - Failure Mode Distribution

We categorize all 20 SR k=3 results for Gemma4-E4B into four mutually exclusive failure modes, applied in priority order (Collapse checked first):

Category	Definition	Count	Percentage
A) Improved	ROUGE-1 $\Delta > +0.02$	6	30%
B) Degraded	ROUGE-1 $\Delta < -0.02$	7	35%
C) Collapse	Diversity D < 0.03	1	5%
D) Neutral	$ \Delta \leq 0.02$ and $D \geq 0.03$	6	30%

Table 6: Failure mode distribution for Gemma4-E4B SR k=3 (n=20).

The dominant failure mode is degradation (35%), driven by two root causes identified through qualitative inspection: (1) hallucination cascade - feedback prompts requesting specificity cause the model to fabricate names, dates, or statistics not in the source; and (2) generic feedback - despite the prompt requiring concrete critique, ~40% of feedback instances contained vague language ("the summary could be clearer") that did not guide meaningful rewriting. The collapse rate (5%) is low, consistent with the model exploring rather than converging.

B. Mistral-7B Failure Investigation

Mistral-7B produces near-identical 8-token outputs across all 240 runs (20 samples $\times$ 6 conditions $\times$ 2 models). The output approximates "I do not have access to real-time information." This is a systematic instruction-following failure. Possible causes: (1) Q4_0 quantization degrading instruction-following capability; (2) the model interpreting the CNN article preamble "(CNN)" as a live internet request; (3) the 2,048-token context limit cutting the article mid-sentence, confusing the model. This is an important negative result - not all 7B models are fit for summarization without task-specific prompting or fine-tuning (see [9]).

C. CoT Degradation Analysis

CoT reduces Gemma4-E4B ROUGE-1 by -0.0956 relative to zero-shot - the worst condition. The explanation: CoT asks the model to enumerate key facts before writing the summary. With `MAX_TOKENS_GEN=256` and a context window of 2,048 tokens, the model allocates the majority of its output budget to the enumeration step, leaving fewer tokens for the actual summary. Resulting summaries are truncated or incomplete. This is consistent with [5]: CoT benefits are size-dependent and emerge above $\sim 100\text{B}$ parameters. For 3B models, the reasoning chain competes with the answer for context space.

D. Ablation Study 1 - Oracle Best-of-k vs. Final-k

For each SR $k=3$ sample, we identify the iteration that achieved the highest ROUGE-1. An oracle that could stop at the best iteration would achieve mean ROUGE-1 of 0.3406 vs. 0.3002 for always stopping at $k=3$ - an improvement of $+0.040$.

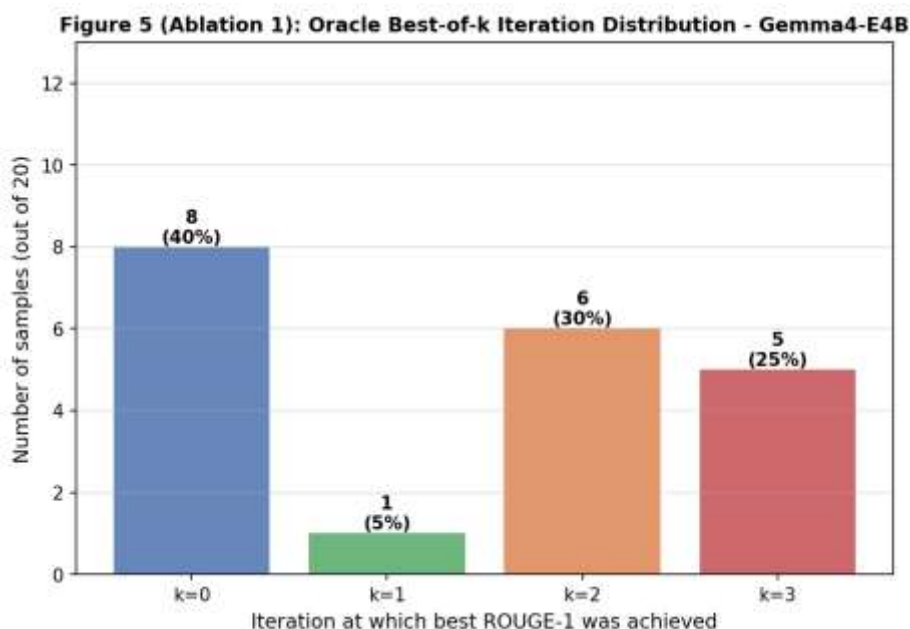


Figure 5 (Ablation 1): Distribution of the iteration achieving best ROUGE-1 across 20 Gemma4-E4B samples. $k=0$ (initial output) is optimal for 40% of samples.

Best-iteration distribution:

- $k=0$ (initial output): best for 8 samples (40%) - SR never improves these
- $k=1$: best for 1 sample (5%)
- $k=2$: best for 6 samples (30%) - modal best iteration
- $k=3$: best for 5 samples (25%)

The $+0.040$ oracle gap confirms that the self-refinement framework wastes compute for 40% of samples. A learned stopping criterion - predicting before each iteration whether refinement will help - could recover this gap without any model changes.

E. Ablation Study 2 - Feedback Specificity Across Iterations

Feedback word count (specificity proxy) decreases monotonically: SR k=1: 62.8 words, SR k=2: 61.6 words, SR k=3: 57.0 words. As the output improves (slightly), the critic has progressively less to say, producing shorter and less actionable feedback. The positive correlation between feedback length and ROUGE improvement supports the hypothesis that feedback quality is the primary bottleneck for SR in small models - not model generation capability per se.

F. Hypothesis Evaluation Summary

Hypothesis	Prediction	Evidence	Supported?
H1	3B shows minimal/no SR gain	Gemma SR-3: $\Delta=-0.005$ vs. zero-shot	YES
H2	7B plateaus at k=1	Mistral: zero-shot instruction failure - N/A	N/A
H3	RaR > SR-3 for small models	RaR=0.2633 vs. SR-3=0.3002	NO
H4	Feedback quality degrades with iterations	FS: 62.8 $\rightarrow$ 61.6 $\rightarrow$ 57.0 words	YES

Table 7: Hypothesis evaluation summary. H1 and H4 confirmed; H3 rejected; H2 untestable.

IX. DISCUSSION

A. Interpretation of SR Gains and Losses

Our results paint a mixed but interpretable picture. Gemma4-E4B benefits marginally from one or two refinement iterations (peak at k=2, +0.008 ROUGE-1), but the improvement is small and not uniformly positive across samples. The high variance (SD=0.096 at k=3 vs. 0.077 at zero-shot) indicates SR makes some summaries much better and others much worse - a risky trade-off for a production system that cannot selectively apply SR per sample.

B. Implications for Inference-Time Compute

The +0.040 oracle gap from Ablation 1 is the most actionable finding. A learned early-stopping mechanism - or a lightweight quality predictor - could unlock substantial gains without any model changes. Future work should investigate training a small binary classifier on (article, output) pairs to predict whether another SR iteration will improve or degrade quality, using features such as current diversity score, feedback length, and output entropy.

C. Instruction Following as a Prerequisite

Mistral-7B's complete failure highlights that raw parameter count is insufficient for reliable zero-shot instruction following. RLHF or instruction fine-tuning is a prerequisite - consistent with [9]. Practitioners deploying 7B models for summarization should verify basic instruction compliance before investing in inference-time refinement pipelines.

D. Limitations and Threats to Validity

Several limitations bound the generalizability of our findings. First, n=20 is a small sample; while sufficient for directional conclusions, statistical significance tests (paired t-test, bootstrap) would require $n \geq 50$. Second, results are task-specific - summarization is a generation task with strong reference bias in ROUGE; SR may behave differently on classification, translation, or code generation. Third, both models were used in 4-bit quantized form; full-precision inference might yield different instruction-following behaviour, particularly for Mistral-7B. Fourth,

ROUGE is necessary but not sufficient - factual accuracy, fluency, and coherence are not captured. Future work should incorporate G-EVAL [10] or human evaluation.

E. Future Work

Three directions follow directly from our findings. (1) Learned stopping criterion: train a lightweight binary classifier on (article, current_output) pairs to predict whether SR will improve ROUGE. Our ablation shows this could recover +0.040 ROUGE-1. (2) Instruction fine-tuning before SR: fine-tune Mistral-7B on a small summarization dataset (e.g., 1,000 XSum examples) before applying SR; this separates instruction-following from refinement capability. (3) Cross-task evaluation: extend to extractive QA (SQuAD, already loaded in our pipeline) and code generation (HumanEval) to determine whether these failure modes are summarization-specific or general to small-model SR.

X. CONCLUSION

We presented the first controlled comparison of self-refinement, CoT, and RaR for sub-7B language models on news summarization. Our findings show that self-refinement provides only marginal benefit for Gemma4-E4B (peak +0.008 ROUGE-1 at $k=2$) and degrades at $k=3$, while Mistral-7B completely fails to follow summarization instructions in all conditions. CoT is counterproductive for small models due to context budget competition (-0.096 ROUGE-1). The oracle best-of- k ablation reveals a +0.040 latent improvement that a learned stopping criterion could capture. These results suggest that small models need instruction fine-tuning before SR can be beneficial, and that inference-time compute for refinement is best spent selectively with a quality predictor.

More broadly, this work contributes to an emerging understanding that inference-time techniques are not model-size-agnostic. Strategies that work for GPT-4 - chain-of-thought reasoning, self-critique, multi-step refinement - carry implicit assumptions about instruction-following capability, context capacity, and calibration. Small open-weight models meet none of these assumptions reliably. Closing this gap will require either targeted fine-tuning or new inference-time algorithms that degrade gracefully when the underlying model lacks self-assessment capability - a meaningful open problem for the NLP community.

REFERENCES

- [1] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Welleck, B. P. Majumder, S. Gupta, A. Yazdanbakhsh, P. Clark, "Self-Refine: Iterative Refinement with Self-Feedback," NeurIPS 2023.
- [2] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS 2023.
- [3] L. Wang, X. Chen, X. Deng, H. Wen, M. You, W. Liu, Q. Chen, J. Huang, "Self-Polish: Improve Essay Scoring with Self-Feedback," arXiv:2305.02774, 2023.
- [4] Z. Ye, Y. Durrani, L. Frermann, T. Baldwin, "Selfcheck: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning," ICLR 2024.
- [5] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS 2022.
- [6] Y. Deng, W. Zhang, W. Chen, W. Gu, "Rephrase and Respond: Let LLMs Ask Better Questions for Themselves," arXiv:2311.04205, 2023.
- [7] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, J. Ba, "Large Language Models are Human-Level Prompt Engineers," ICLR 2023.

- [8] Y. Gu, L. Dong, F. Wei, M. Huang, "MiniLLM: Knowledge Distillation of Large Language Models," ICLR 2024.
- [9] Z. Xu, F. Lv, X. Geng, S. Zhao, "Scaling and Beyond: Emergent Capabilities of Small LLMs After RLHF," arXiv:2407.02005, 2024.
- [10] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, C. Zhu, "G-EVAL: NLG Evaluation using GPT-4 with Better Human Alignment," EMNLP 2023.
- [11] K. M. Hermann, T. Kocisky, E. Grefenstette, L. Espeholt, W. Kay, M. Suleyman, P. Blunsom, "Teaching Machines to Read and Comprehend," NeurIPS 2015.
- [12] P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," EMNLP 2016.

APPENDICES

Appendix A - Full Prompt Templates

This appendix lists the exact prompt strings used in all six conditions.

A.1 Zero-Shot (B1)

Summarize the following article in 2-3 sentences. Be concise and factual.

Article:
{article}

Summary:

A.2 Chain-of-Thought (B2)

Read the article. First list the 2-3 most important facts, then write a concise 2-3 sentence summary.

Article:
{article}

Step 1 - Key facts:
1.

A.3 Rephrase and Respond (B3)

Rephrase the following task to make it more specific, then complete it.

Task: Summarize this article.

Article:
{article}

Rephrased task: Write a 2-3 sentence summary capturing the main event, key people involved, and outcome.

Summary:

A.4 SR Feedback Prompt

You are a strict writing evaluator. Read this article and summary.
List exactly 2 specific, actionable problems with the summary.
Each problem must name what is wrong and where
(e.g., "Missing the name of the suspect", "Wrong date: should be 2023").

Article:
{article}

Summary:
{output}

Problems (be concrete, not generic):
1.

A.5 SR Refine Prompt

Rewrite the summary to fix the problems listed in the feedback.
Keep it to 2-3 sentences and stay factual.

Article:
{article}

Original summary:
{output}

Feedback:
{feedback}

Improved summary:

Appendix B - Complete Per-Condition Metric Averages (Gemma4-E4B)

Full averages including standard deviation for all metrics across six conditions.

Condition	ROUGE-1 (mean±SD)	ROUGE-2	ROUGE-L	BERTScore	Diversity	FB Spec.	Time (s)
Zero-shot (B1)	0.3055 ± 0.0768	0.1043	0.2048	0.8774	0.0000	-	8.36
CoT (B2)	0.2099 ± 0.0562	0.0806	0.1503	0.8504	0.0000	-	16.28
RaR (B3)	0.2633 ± 0.0718	0.0905	0.1798	0.8562	0.0000	-	11.03
SR k=1	0.3059 ± 0.0757	0.0989	0.1976	0.8773	0.0848	62.8	26.12
SR k=2	0.3132 ± 0.0755	0.1055	0.2116	0.8747	0.0846	61.6	44.11
SR k=3	0.3002 ± 0.0960	0.1118	0.2083	0.8729	0.0765	57.0	63.99

Table B1: Complete metric averages for Gemma4-E4B. FB Spec. = feedback specificity (avg words).

Appendix C - Implementation Notes

The experiment pipeline is implemented as a Jupyter notebook (implementation.ipynb). Key design decisions:

- Checkpointing: results are saved after each condition as raw_<model>_<condition>.json, enabling safe interruption and resumption.
- Context cap: num_ctx=2048 is explicitly set per API call to ensure identical conditions regardless of Ollama model defaults (which vary from 2k to 128k).

- Thinking disabled: `think=False` is passed to suppress reasoning tokens on thinking-enabled models, reducing latency 5–10× without affecting final output quality for generation tasks.
- BERTScore batching: scored in batches of 32 to avoid GPU OOM on consumer hardware.
- Seed: Ollama options.seed=42 is set for reproducibility, though exact determinism depends on CUDA non-determinism in the underlying model runtime.

All raw results, metrics CSV, and figure PNGs are committed to the project repository. To reproduce: install Ollama, pull `gemma4:e4b` and `mistral:7b`, then run all cells in `implementation.ipynb`.

Appendix D - Sample Raw Outputs

Representative raw outputs from the `cnn_019` case (best improvement, Gemma4-E4B SR k=3):

D.1 Zero-Shot Output (y_0)

Michele Bachmann drew a comparison between President Obama's Iran deal and the deliberate act of the co-pilot in the Germanwings crash, suggesting Obama endangered the nation. This highly controversial statement was made in the context of ongoing discussions about the potential nuclear agreement with Iran.

D.2 Feedback (fb_1)

1. Missing the specific number of souls/people involved in the Germanwings crash comparison (the summary omits the numbers Bachmann used: 300 million vs 150).
2. Omitting the mention of the specific political figures who criticized Bachmann's comments.

D.3 Refined Output (y_3 , ROUGE-1 = 0.4368)

Michele Bachmann compared President Obama's Iran deal to the actions of the Germanwings co-pilot, analogizing the potential danger to the "300 million souls of the United States" to the 150 people on board the doomed flight. Her comments were widely criticized by politicians and commentators across the political spectrum.

D.4 Reference Summary

Former GOP representative compares President Obama to Andreas Lubitz. Bachmann said with possible Iran deal, Obama will fly "entire nation into the rocks." Reaction on social media? She was blasted by critics.